Event Management System

Database Management System (DBMS) Project

Subject	Database Management System (DBMS) — UCS4001
Topic	Event Management System
Language	SQL (MySQL Compatible)
Submitted By	Vansh Sharma — 2410030122
	Himanshi Kumar — 2410030114
	Ananya Goel — 2410030071
	Geetanshi Singh — 2410030068
	Kushagra Goyal — 2410030112
Department	Computer Science & Engineering
Institution	IILM University, Greater Noida
Academic Year	2025 – 2026
Submitted To	Dr. Neeraj Pratap Sir

Event Management System – Database Design (DBMS Project)

Introduction:

An Event Management System (EMS) allows organizers to create and manage events, register users, handle venue bookings, and process payments online. The database stores information about:

- Users (Attendees & Organizers)
- Events
- Categories
- Venues
- Registrations
- Payments

The system ensures **data consistency, security, and efficient transaction management** for all event-related operations.

ENTITIES AND TABLES :-

1. Users Table

Stores attendee and organizer user details.

Field Name	Data Type	Description
user_id (PK)	INT	Unique user ID
full_name	VARCHAR(100)	Full name of the user
email	VARCHAR(150)	Unique email address
phone	VARCHAR(15)	Contact number
city	VARCHAR(80)	City of residence
created_at	DATETIME	Account creation timestamp

---

2. Categories Table

Stores event type/genre categories.

Field Name	Data Type
category_id (PK)	INT
category_name	VARCHAR(80)
description	TEXT

---

3. Venues Table

Stores physical or virtual locations for events.

Field Name	Data Type	Description
venue_id (PK)	INT	Unique venue ID
venue_name	VARCHAR(120)	Name of the venue
address	VARCHAR(200)	Street address
city	VARCHAR(80)	City of the venue
capacity	INT	Maximum seating capacity
contact_phone	VARCHAR(15)	Venue contact number

4. Organizers Table

Stores details of organizations or individuals who host events.

Field Name	Data Type	Description
organizer_id (PK)	INT	Unique organizer ID
org_name	VARCHAR(120)	Organization name
contact_person	VARCHAR(100)	Primary contact person
email	VARCHAR(150)	Unique organizer email
phone	VARCHAR(15)	Contact phone number

5. Events Table

Central table storing all event details.

Field Name	Data Type	Description
event_id (PK)	INT	Unique event ID
event_name	VARCHAR(150)	Name/title of the event
category_id	INT	Category of the event
organizer_id	INT	Organizer of the event
venue_id	INT	Venue of the event
event_date	DATE	Date of the event
start_time	TIME	Event start time
end_time	TIME	Event end time
description	TEXT	Event description
ticket_price	DECIMAL(10,2)	Ticket price (0 = free)
max_capacity	INT	Maximum attendees allowed
status	ENUM	'Upcoming','Ongoing','Completed','Cancelled'

6. Registrations Table

Records user registrations for events.

Field Name	Data Type	Description
reg_id (PK)	INT	Unique registration ID
user_id	INT	ID of the registered user
event_id	INT	ID of the event
reg_date	DATETIME	Date and time of registration
ticket_count	INT	Number of tickets booked
status	ENUM	'Confirmed','Pending','Cancelled'

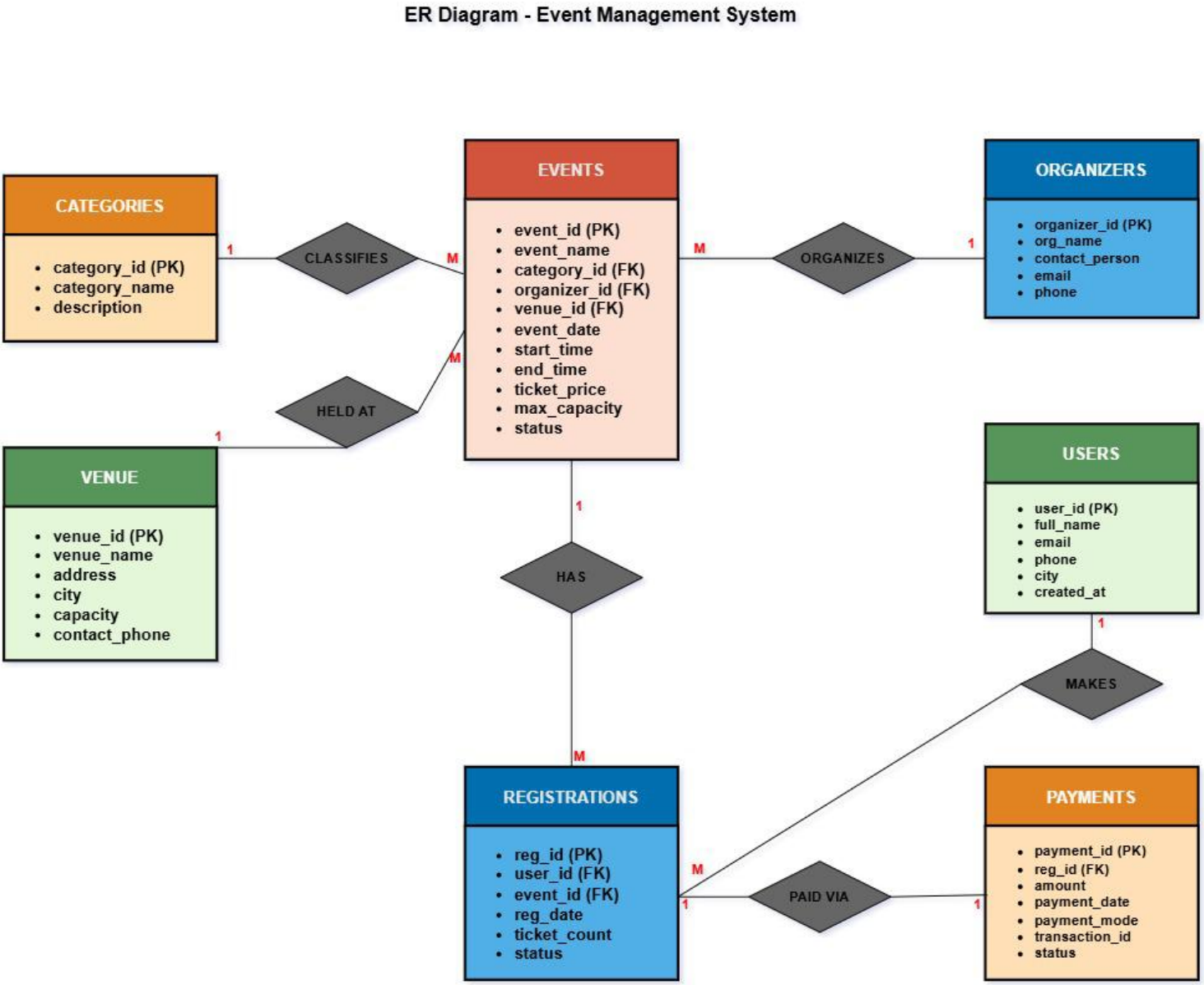
7. Payments Table

Tracks payment details linked to registrations.

Field Name	Data Type	Description
payment_id (PK)	INT	Unique payment ID
reg_id	INT	Linked registration ID
amount	DECIMAL(10,2)	Payment amount
payment_date	DATETIME	Date and time of payment
payment_mode	ENUM	'Cash','Card','UPI','Net Banking'
transaction_id	VARCHAR(100)	Unique transaction reference
status	ENUM	'Success','Failed','Refunded'

ER Diagram Representation :-

The Entity-Relationship (ER) diagram below depicts the entities, their attributes, and the relationships among them. Blue rectangles represent entities, ovals represent attributes, and yellow diamonds represent relationships between entities.



ER DIAGRAM STRUCTURE :-

User (1) ———————— (M) Registrations

Event (1) ———————— (M) Registrations

Registrations (1) ——— (1) Payments

Category (1) ———————— (M) Events

Organizer (1) ———————— (M) Events

Venue (1) ———————— (M) Events

SQL TABLE CREATION :-

```sql
CREATE DATABASE IF NOT EXISTS event_management;
USE event_management;
-- 1. CATEGORIES
CREATE TABLE categories (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(80) NOT NULL UNIQUE,
    description TEXT
);
-- 2. VENUES
CREATE TABLE venues (
    venue_id INT PRIMARY KEY,
    venue_name VARCHAR(120) NOT NULL,
    address VARCHAR(200),
    city VARCHAR(80),
    capacity INT,
    contact_phone VARCHAR(15)
);
-- 3. ORGANIZERS
CREATE TABLE organizers (
    organizer_id INT PRIMARY KEY,
    org_name VARCHAR(120) NOT NULL,
    contact_person VARCHAR(100),
    email VARCHAR(150) UNIQUE,
    phone VARCHAR(15)
);
-- 4. USERS
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    phone VARCHAR(15),
    city VARCHAR(80),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
-- 5. EVENTS
CREATE TABLE events (
    event_id INT PRIMARY KEY,
    event_name VARCHAR(150) NOT NULL,
    category_id INT,
    organizer_id INT,
    venue_id INT,
    event_date DATE NOT NULL,
    start_time TIME,
    end_time TIME,
    description TEXT,
    ticket_price DECIMAL(10,2) DEFAULT 0.00,
```

```
    max_capacity INT,
    status ENUM('Upcoming','Ongoing','Completed','Cancelled') DEFAULT 'Upcoming'
);
-- 6. REGISTRATIONS
CREATE TABLE registrations (
    reg_id INT PRIMARY KEY,
    user_id INT NOT NULL,
    event_id INT NOT NULL,
    reg_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    ticket_count INT DEFAULT 1,
    status ENUM('Confirmed','Pending','Cancelled') DEFAULT 'Confirmed'
);
-- 7. PAYMENTS
CREATE TABLE payments (
    payment_id INT PRIMARY KEY,
    reg_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    payment_mode ENUM('Cash','Card','UPI','Net Banking'),
    transaction_id VARCHAR(100) UNIQUE,
    status ENUM('Success','Failed','Refunded') DEFAULT 'Success'
);
```

Data Insert Queries :-

Insert Categories:

```
INSERT INTO categories (category_name, description) VALUES
('Conference', 'Professional conferences and seminars'),
('Wedding', 'Wedding ceremonies and receptions'),
('Concert', 'Music concerts and live performances'),
('Sports', 'Sporting events and tournaments'),
('Workshop', 'Hands-on skill development workshops'),
('Festival', 'Cultural and food festivals'),
('Corporate', 'Corporate meetings and launches');
```

Insert Venues:

```
INSERT INTO venues (venue_name, address, city, capacity, contact_phone) VALUES
('Grand Ballroom', '12 Park Street', 'Mumbai', 2000, '9821000001'),
('Tech Hub Arena', '45 IT Park, Whitefield', 'Bangalore', 1500, '9845000002'),
('City Convention Centre', '78 MG Road', 'Delhi', 3000, '9811000003'),
('Riverside Open Ground', 'Near River Bridge', 'Pune', 5000, '9765000004'),
('Expo Pavilion', 'Bandra-Kurla Complex', 'Mumbai', 4000, '9821000005');
```

Insert Organizers:

```
INSERT INTO organizers (org_name, contact_person, email, phone) VALUES
('StarEvents Pvt Ltd', 'Priya Sharma', 'priya@starevents.com', '9901111111'),
('TechConf India', 'Arjun Mehta', 'arjun@techconf.in', '9902222222'),
('Royal Celebrations', 'Sunita Raut', 'sunita@royalcel.com', '9903333333'),
('Beat & Beats Music', 'Rohan Das', 'rohan@beatbeats.com', '9904444444');
```

Insert Users:

```
INSERT INTO users (full_name, email, phone, city) VALUES
('Aarav Kumar', 'aarav.kumar@gmail.com', '9810001111', 'Delhi'),
('Sneha Patel', 'sneha.patel@yahoo.com', '9820002222', 'Mumbai'),
('Rohit Sharma', 'rohit.sharma@outlook.com', '9830003333', 'Bangalore'),
('Anjali Singh', 'anjali.singh@gmail.com', '9840004444', 'Pune'),
('Vikas Gupta', 'vikas.gupta@hotmail.com', '9850005555', 'Chennai'),
('Pooja Nair', 'pooja.nair@gmail.com', '9860006666', 'Mumbai'),
('Karan Mehta', 'karan.mehta@gmail.com', '9870007777', 'Delhi'),
('Divya Reddy', 'divya.reddy@gmail.com', '9880008888', 'Hyderabad');
```

Insert Events:

```
INSERT INTO events (event_name, category_id, organizer_id, venue_id,
    event_date, start_time, end_time, description, ticket_price, max_capacity, status)
VALUES
('India Tech Summit 2025', 1, 2, 2, '2025-03-15', '09:00:00', '18:00:00',
    'Annual technology summit', 1500.00, 1500, 'Completed'),
('Raj & Priya Wedding', 2, 3, 1, '2025-04-20', '18:00:00', '23:00:00',
    'Grand wedding ceremony', 0.00, 500, 'Completed'),
('Arijit Singh Live Concert', 1, 4, 5, '2025-05-10', '19:00:00', '22:00:00',
    'Live music concert', 2500.00, 3500, 'Completed'),
```

```
('Python Workshop Series', 5, 2, 3, '2025-07-05', '10:00:00', '17:00:00',
    'Python programming workshop', 800.00, 200, 'Upcoming'),
('Mumbai Food Festival', 6, 1, 5, '2025-07-20', '11:00:00', '21:00:00',
    'Multi-cuisine food festival', 300.00, 3500, 'Upcoming');
```

Insert Registrations:

```
INSERT INTO registrations (user_id, event_id, ticket_count, status) VALUES
(1, 1, 1, 'Confirmed'),
(2, 1, 2, 'Confirmed'),
(3, 1, 1, 'Confirmed'),
(4, 2, 4, 'Confirmed'),
(5, 2, 2, 'Confirmed'),
(1, 3, 2, 'Confirmed'),
(6, 3, 1, 'Confirmed'),
(7, 3, 3, 'Confirmed'),
(4, 4, 1, 'Pending'),
(5, 5, 3, 'Confirmed');
```

Insert Payments:

```
INSERT INTO payments (reg_id, amount, payment_mode, transaction_id, status) VALUES
(1, 1500.00, 'UPI', 'TXN20250001', 'Success'),
(2, 3000.00, 'Card', 'TXN20250002', 'Success'),
(3, 1500.00, 'Net Banking', 'TXN20250003', 'Success'),
(4, 0.00, 'Cash', 'TXN20250004', 'Success'),
(5, 0.00, 'Cash', 'TXN20250005', 'Success'),
(6, 5000.00, 'Card', 'TXN20250006', 'Success'),
(7, 2500.00, 'UPI', 'TXN20250007', 'Success'),
(8, 7500.00, 'Net Banking', 'TXN20250008', 'Success'),
(10, 900.00, 'UPI', 'TXN20250009', 'Success');
```

SQL Select Queries :-

- **List all upcoming events with venue and category**

```
SELECT e.event_name, c.category_name, v.venue_name,
    v.city, e.event_date, e.ticket_price
FROM events e
JOIN categories c ON e.category_id = c.category_id
JOIN venues v ON e.venue_id = v.venue_id
WHERE e.status = 'Upcoming'
ORDER BY e.event_date;
```

- **Count registrations per event**

```
SELECT e.event_name, COUNT(r.reg_id) AS total_registrations,
    SUM(r.ticket_count) AS total_tickets
FROM events e
LEFT JOIN registrations r ON e.event_id = r.event_id
    AND r.status = 'Confirmed'
GROUP BY e.event_id, e.event_name
ORDER BY total_tickets DESC;
```

- **Total revenue per event**

```
SELECT e.event_name,
    SUM(p.amount) AS total_revenue,
    COUNT(p.payment_id) AS payment_count
FROM events e
JOIN registrations r ON e.event_id = r.event_id
JOIN payments p ON r.reg_id = p.reg_id
WHERE p.status = 'Success'
GROUP BY e.event_id, e.event_name
ORDER BY total_revenue DESC;
```

- **Users registered for more than one event**

```
SELECT u.full_name, u.email, COUNT(r.event_id) AS events_registered
FROM users u
JOIN registrations r ON u.user_id = r.user_id
WHERE r.status = 'Confirmed'
GROUP BY u.user_id, u.full_name, u.email
HAVING COUNT(r.event_id) > 1
ORDER BY events_registered DESC;
```

- **Events with available capacity**

```
SELECT e.event_name, e.max_capacity,
    COALESCE(SUM(r.ticket_count), 0) AS booked,
    e.max_capacity - COALESCE(SUM(r.ticket_count), 0) AS available
FROM events e
LEFT JOIN registrations r
    ON e.event_id = r.event_id AND r.status = 'Confirmed'
GROUP BY e.event_id, e.event_name, e.max_capacity
HAVING available > 0
ORDER BY available DESC;
```

- **Payment mode breakdown**

```
SELECT payment_mode,
    COUNT(*) AS transactions,
    SUM(amount) AS total_amount
FROM payments
WHERE status = 'Success'
GROUP BY payment_mode
ORDER BY total_amount DESC;
```

Conclusion :-

This project successfully covers the design and implementation of a database for an Event Management System using SQL. The database includes 7 tables — Categories, Venues, Organizers, Users, Events, Registrations, and Payments — which together manage all the important information related to events.

The system makes it easy to store and retrieve data about events, registered users, venue details, and payment records. Overall, this project gave a good understanding of how a real-world database is designed and used.

SQL Queries and Outputs

Q1. Retrieve all records from the categories table.

```
MariaDB [event_management1]> SELECT * FROM categories;
+-------------+---------------+-----------------------------+
| category_id | category_name | description                 |
+-------------+---------------+-----------------------------+
|           1 | Music         | Concerts and music festivals |
|           2 | Tech          | Tech events and hackathons   |
|           3 | Sports        | Sports competitions          |
|           4 | Education     | Workshops and seminars       |
|           5 | Art           | Art exhibitions              |
|           6 | Business      | Corporate events             |
|           7 | Health        | Fitness and wellness events  |
|           8 | Gaming        | Gaming tournaments           |
|           9 | Fashion       | Fashion shows                |
|          10 | Food          | Food festivals               |
+-------------+---------------+-----------------------------+
10 rows in set (0.001 sec)
```

Q2. Retrieve all records from the events table.

```
MariaDB [event_management1]> SELECT * FROM events;
+----------+------------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
| event_id | event_name       | category_id | organizer_id | venue_id | event_date | start_time | end_time | description | ticket_price | max_capacity | status   |
+----------+------------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
|        1 | Rock Concert     |           1 |            1 |        1 | 2026-04-01 | 18:00:00   | 22:00:00 | NULL        |       999.00 |          500 | Upcoming |
|        2 | Hackathon        |           2 |            2 |        2 | 2026-04-05 | 09:00:00   | 18:00:00 | NULL        |         0.00 |          300 | Upcoming |
|        3 | Football Match   |           3 |            3 |        3 | 2026-04-10 | 16:00:00   | 19:00:00 | NULL        |       500.00 |         1000 | Upcoming |
|        4 | Workshop AI      |           4 |            4 |        4 | 2026-04-12 | 10:00:00   | 14:00:00 | NULL        |       200.00 |          400 | Upcoming |
|        5 | Art Expo         |           5 |            5 |        5 | 2026-04-15 | 11:00:00   | 17:00:00 | NULL        |       150.00 |          200 | Upcoming |
|        6 | Startup Meet     |           6 |            6 |        6 | 2026-04-18 | 10:00:00   | 15:00:00 | NULL        |       300.00 |          350 | Upcoming |
|        7 | Yoga Camp        |           7 |            7 |        7 | 2026-04-20 | 06:00:00   | 09:00:00 | NULL        |       100.00 |          250 | Upcoming |
|        8 | Gaming Tournament |          8 |            8 |        8 | 2026-04-22 | 12:00:00   | 20:00:00 | NULL        |       250.00 |          150 | Upcoming |
|        9 | Fashion Show     |           9 |            9 |        9 | 2026-04-25 | 17:00:00   | 21:00:00 | NULL        |       800.00 |          300 | Upcoming |
|       10 | Food Festival    |          10 |           10 |       10 | 2026-04-28 | 12:00:00   | 22:00:00 | NULL        |       100.00 |          600 | Upcoming |
+----------+------------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
10 rows in set (0.004 sec)
```

Q3. Display event names along with their ticket prices.

```
MariaDB [event_management1]> SELECT event_name, ticket_price FROM events;
+------------------+--------------+
| event_name       | ticket_price |
+------------------+--------------+
| Rock Concert     |       999.00 |
| Hackathon        |         0.00 |
| Football Match   |       500.00 |
| Workshop AI      |       200.00 |
| Art Expo         |       150.00 |
| Startup Meet     |       300.00 |
| Yoga Camp        |       100.00 |
| Gaming Tournament |      250.00 |
| Fashion Show     |       800.00 |
| Food Festival    |       100.00 |
+------------------+--------------+
10 rows in set (0.001 sec)
```

Q4. Retrieve all users who belong to Delhi.

```
MariaDB [event_management1]> SELECT * FROM users WHERE city = 'Delhi';
+---------+--------------+-----------------+------------+-------+---------------------+
| user_id | full_name    | email           | phone      | city  | created_at          |
+---------+--------------+-----------------+------------+-------+---------------------+
|       2 | Aman Gupta   | aman@gmail.com  | 9991110002 | Delhi | 2026-03-26 18:20:38 |
|       3 | Priya Sharma | priya@gmail.com | 9991110003 | Delhi | 2026-03-26 18:20:38 |
|       5 | Sneha Kapoor | sneha@gmail.com | 9991110005 | Delhi | 2026-03-26 18:20:38 |
|       7 | Neha Arora   | neha@gmail.com  | 9991110007 | Delhi | 2026-03-26 18:20:38 |
|       8 | Vikas Jain   | vikas@gmail.com | 9991110008 | Delhi | 2026-03-26 18:20:38 |
|      10 | Rahul Mehta  | rahul@gmail.com | 9991110010 | Delhi | 2026-03-26 18:20:38 |
+---------+--------------+-----------------+------------+-------+---------------------+
6 rows in set (0.011 sec)
```

Q5. Retrieve all events where ticket price is greater than 500.

```
MariaDB [event_management1]> SELECT * FROM events WHERE ticket_price > 500;
+---------+-------------+-------------+-------------+---------+-----------+-----------+---------+-------------+-------------+-------------+---------+
| event_id | event_name  | category_id | organizer_id | venue_id | event_date | start_time | end_time | description | ticket_price | max_capacity | status  |
+---------+-------------+-------------+-------------+---------+-----------+-----------+---------+-------------+-------------+-------------+---------+
|       1 | Rock Concert |           1 |           1 |       1 | 2026-04-01 | 18:00:00  | 22:00:00 | NULL        |      999.00 |         500 | Upcoming |
|       9 | Fashion Show |           9 |           9 |       9 | 2026-04-25 | 17:00:00  | 21:00:00 | NULL        |      800.00 |         300 | Upcoming |
+---------+-------------+-------------+-------------+---------+-----------+-----------+---------+-------------+-------------+-------------+---------+
2 rows in set (0.001 sec)
```

Q6. Retrieve all registrations with status "Confirmed".

```
MariaDB [event_management1]> SELECT * FROM registrations WHERE status = 'Confirmed';
+--------+---------+----------+---------------------+--------------+-----------+
| reg_id | user_id | event_id | reg_date            | ticket_count | status    |
+--------+---------+----------+---------------------+--------------+-----------+
|      1 |       1 |        1 | 2026-03-26 18:20:59 |            2 | Confirmed |
|      2 |       2 |        2 | 2026-03-26 18:20:59 |            1 | Confirmed |
|      4 |       4 |        4 | 2026-03-26 18:20:59 |            1 | Confirmed |
|      6 |       6 |        6 | 2026-03-26 18:20:59 |            1 | Confirmed |
|      7 |       7 |        7 | 2026-03-26 18:20:59 |            2 | Confirmed |
|      9 |       9 |        9 | 2026-03-26 18:20:59 |            2 | Confirmed |
|     10 |      10 |       10 | 2026-03-26 18:20:59 |            4 | Confirmed |
+--------+---------+----------+---------------------+--------------+-----------+
7 rows in set (0.010 sec)
```

Q7. Retrieve all payments with status "Success".

```
MariaDB [event_management1]> SELECT * FROM payments WHERE status = 'Success';
+------------+--------+---------+---------------------+--------------+----------------+---------+
| payment_id | reg_id | amount  | payment_date        | payment_mode | transaction_id | status  |
+------------+--------+---------+---------------------+--------------+----------------+---------+
|          1 |      1 | 1998.00 | 2026-03-26 18:21:09 | UPI          | TXN001         | Success |
|          2 |      2 |    0.00 | 2026-03-26 18:21:09 | Cash         | TXN002         | Success |
|          4 |      4 |  200.00 | 2026-03-26 18:21:09 | UPI          | TXN004         | Success |
|          6 |      6 |  300.00 | 2026-03-26 18:21:09 | UPI          | TXN006         | Success |
|          7 |      7 |  200.00 | 2026-03-26 18:21:09 | Cash         | TXN007         | Success |
|          9 |      9 | 1600.00 | 2026-03-26 18:21:09 | UPI          | TXN009         | Success |
|         10 |     10 |  400.00 | 2026-03-26 18:21:09 | Cash         | TXN010         | Success |
+------------+--------+---------+---------------------+--------------+----------------+---------+
7 rows in set (0.011 sec)
```

Q8. Retrieve all venues with capacity greater than 300.

```
MariaDB [event_management1]> SELECT * FROM venues WHERE capacity > 300;
+----------+--------------+----------------+---------+----------+---------------+
| venue_id | venue_name   | address        | city    | capacity | contact_phone |
+----------+--------------+----------------+---------+----------+---------------+
|        1 | City Hall    | MG Road        | Delhi   |      500 | 9000000001    |
|        3 | Stadium A    | Sports Complex | Delhi   |     1000 | 9000000003    |
|        4 | Auditorium X | Central Delhi  | Delhi   |      400 | 9000000004    |
|        6 | Biz Center   | Gurgaon        | Gurgaon |      350 | 9000000006    |
|       10 | Food Court   | Karol Bagh     | Delhi   |      600 | 9000000010    |
+----------+--------------+----------------+---------+----------+---------------+
5 rows in set (0.011 sec)
```

Q9. Count the total number of users.

```
MariaDB [event_management1]> SELECT COUNT(*) AS total_users FROM users;
+--------------+
| total_users |
+--------------+
|           10 |
+--------------+
1 row in set (0.014 sec)
```

Q10. Find the average ticket price of all events.

```
MariaDB [event_management1]> SELECT AVG(ticket_price) AS avg_price FROM events;
+------------+
| avg_price  |
+------------+
| 339.900000 |
+------------+
1 row in set (0.001 sec)
```

Q11. Find the highest ticket price among all events.

```
MariaDB [event_management1]> SELECT MAX(ticket_price) AS highest_price FROM events;
+---------------+
| highest_price |
+---------------+
|        999.00 |
+---------------+
1 row in set (0.001 sec)
```

Q12. Find the lowest ticket price among all events.

```
MariaDB [event_management1]> SELECT MIN(ticket_price) AS lowest_price FROM events;
+--------------+
| lowest_price |
+--------------+
|         0.00 |
+--------------+
1 row in set (0.001 sec)
```

Q13. Calculate the total revenue generated from successful payments.

```
MariaDB [event_management1]> SELECT SUM(amount) AS total_revenue FROM payments WHERE status='Success';
+---------------+
| total_revenue |
+---------------+
|       4698.00 |
+---------------+
1 row in set (0.001 sec)
```

Q14. Display event names along with their category names.

```
MariaDB [event_management1]> SELECT e.event_name, c.category_name
    -> FROM events e JOIN categories c ON e.category_id = c.category_id;
+-------------------+---------------+
| event_name        | category_name |
+-------------------+---------------+
| Rock Concert      | Music         |
| Hackathon         | Tech          |
| Football Match    | Sports        |
| Workshop AI       | Education     |
| Art Expo          | Art           |
| Startup Meet      | Business      |
| Yoga Camp         | Health        |
| Gaming Tournament | Gaming        |
| Fashion Show      | Fashion       |
| Food Festival     | Food          |
+-------------------+---------------+
10 rows in set (0.002 sec)
```

Q15. Display event names along with their organizer names.

```
MariaDB [event_management1]> SELECT e.event_name, o.org_name
    -> FROM events e JOIN organizers o ON e.organizer_id = o.organizer_id;
+-------------------+---------------+
| event_name        | org_name      |
+-------------------+---------------+
| Rock Concert      | EventCorp     |
| Hackathon         | TechWorld     |
| Football Match    | SportsHub     |
| Workshop AI       | EduPro        |
| Art Expo          | Artify        |
| Startup Meet      | BizMeet       |
| Yoga Camp         | HealthFirst   |
| Gaming Tournament | GameZone      |
| Fashion Show      | FashionFiesta |
| Food Festival     | Foodies       |
+-------------------+---------------+
10 rows in set (0.012 sec)
```

Q16. Display event names along with their venue names.

```
MariaDB [event_management1]> SELECT e.event_name, v.venue_name
    -> FROM events e JOIN venues v ON e.venue_id = v.venue_id;
+-------------------+---------------+
| event_name        | venue_name    |
+-------------------+---------------+
| Rock Concert      | City Hall     |
| Hackathon         | Tech Park     |
| Football Match    | Stadium A     |
| Workshop AI       | Auditorium X  |
| Art Expo          | Art Gallery   |
| Startup Meet      | Biz Center    |
| Yoga Camp         | Fitness Arena |
| Gaming Tournament | Gaming Hub    |
| Fashion Show      | Fashion Arena |
| Food Festival     | Food Court    |
+-------------------+---------------+
10 rows in set (0.001 sec)
```

Q17. Display user names along with the events they registered for.

```
MariaDB [event_management1]> SELECT u.full_name, e.event_name
    -> FROM registrations r
    -> JOIN users u ON r.user_id = u.user_id
    -> JOIN events e ON r.event_id = e.event_id;
+----------------+-------------------+
| full_name      | event_name        |
+----------------+-------------------+
| Kushagra Goyal | Rock Concert      |
| Aman Gupta     | Hackathon         |
| Priya Sharma   | Football Match    |
| Rohit Verma    | Workshop AI       |
| Sneha Kapoor   | Art Expo          |
| Aditya Singh   | Startup Meet      |
| Neha Arora     | Yoga Camp         |
| Vikas Jain     | Gaming Tournament |
| Simran Kaur    | Fashion Show      |
| Rahul Mehta    | Food Festival     |
+----------------+-------------------+
10 rows in set (0.001 sec)
```

Q18. Display payment amounts along with the corresponding user names.

```
MariaDB [event_management1]> SELECT p.amount, u.full_name
    -> FROM payments p
    -> JOIN registrations r ON p.reg_id = r.reg_id
    -> JOIN users u ON r.user_id = u.user_id;
+---------+----------------+
| amount  | full_name      |
+---------+----------------+
| 1998.00 | Kushagra Goyal |
|    0.00 | Aman Gupta     |
| 1500.00 | Priya Sharma   |
|  200.00 | Rohit Verma    |
|  300.00 | Sneha Kapoor   |
|  300.00 | Aditya Singh   |
|  200.00 | Neha Arora     |
|  250.00 | Vikas Jain     |
| 1600.00 | Simran Kaur    |
|  400.00 | Rahul Mehta    |
+---------+----------------+
10 rows in set (0.001 sec)
```

Q19. Count the number of events in each category.

```
MariaDB [event_management1]> SELECT category_id, COUNT(*) AS total_events
    -> FROM events GROUP BY category_id;
+-------------+--------------+
| category_id | total_events |
+-------------+--------------+
|           1 |            1 |
|           2 |            1 |
|           3 |            1 |
|           4 |            1 |
|           5 |            1 |
|           6 |            1 |
|           7 |            1 |
|           8 |            1 |
|           9 |            1 |
|          10 |            1 |
+-------------+--------------+
10 rows in set (0.006 sec)
```

Q20. Count the number of users in each city.

```
MariaDB [event_management1]> SELECT city, COUNT(*) AS total_users
    -> FROM users GROUP BY city;
+---------+-------------+
| city    | total_users |
+---------+-------------+
| Delhi   |           6 |
| Gurgaon |           1 |
| Noida   |           2 |
| Punjab  |           1 |
+---------+-------------+
4 rows in set (0.002 sec)
```

Q21. Count the number of payments based on their status.

```
MariaDB [event_management1]> SELECT status, COUNT(*) AS total_payments
    -> FROM payments GROUP BY status;
+----------+----------------+
| status   | total_payments |
+----------+----------------+
| Success  |              7 |
| Failed   |              2 |
| Refunded |              1 |
+----------+----------------+
3 rows in set (0.001 sec)
```

Q22. Calculate total tickets booked for each event.

```
MariaDB [event_management1]> SELECT event_id, SUM(ticket_count) AS total_tickets
    -> FROM registrations GROUP BY event_id;
+----------+----------------+
| event_id | total_tickets |
+----------+----------------+
|        1 |              2 |
|        2 |              1 |
|        3 |              3 |
|        4 |              1 |
|        5 |              2 |
|        6 |              1 |
|        7 |              2 |
|        8 |              1 |
|        9 |              2 |
|       10 |              4 |
+----------+----------------+
10 rows in set (0.006 sec)
```

Q23. Retrieve all events sorted by ticket price in descending order.

```
MariaDB [event_management1]> SELECT * FROM events ORDER BY ticket_price DESC;
+----------+------------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
| event_id | event_name       | category_id | organizer_id | venue_id | event_date | start_time | end_time | description | ticket_price | max_capacity | status   |
+----------+------------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
|        1 | Rock Concert     |           1 |            1 |        1 | 2026-04-01 | 18:00:00   | 22:00:00 | NULL        |       999.00 |          500 | Upcoming |
|        9 | Fashion Show     |           9 |            9 |        9 | 2026-04-25 | 17:00:00   | 21:00:00 | NULL        |       800.00 |          300 | Upcoming |
|        3 | Football Match   |           3 |            3 |        3 | 2026-04-10 | 16:00:00   | 19:00:00 | NULL        |       500.00 |         1000 | Upcoming |
|        6 | Startup Meet     |           6 |            6 |        6 | 2026-04-18 | 10:00:00   | 15:00:00 | NULL        |       300.00 |          350 | Upcoming |
|        8 | Gaming Tournament |          8 |            8 |        8 | 2026-04-22 | 12:00:00   | 20:00:00 | NULL        |       250.00 |          150 | Upcoming |
|        4 | Workshop AI      |           4 |            4 |        4 | 2026-04-12 | 10:00:00   | 14:00:00 | NULL        |       200.00 |          400 | Upcoming |
|        5 | Art Expo         |           5 |            5 |        5 | 2026-04-15 | 11:00:00   | 17:00:00 | NULL        |       150.00 |          200 | Upcoming |
|        7 | Yoga Camp        |           7 |            7 |        7 | 2026-04-20 | 06:00:00   | 09:00:00 | NULL        |       100.00 |          250 | Upcoming |
|       10 | Food Festival    |          10 |           10 |       10 | 2026-04-28 | 12:00:00   | 22:00:00 | NULL        |       100.00 |          600 | Upcoming |
|        2 | Hackathon        |           2 |            2 |        2 | 2026-04-05 | 09:00:00   | 18:00:00 | NULL        |         0.00 |          300 | Upcoming |
+----------+------------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
10 rows in set (0.001 sec)
```

Q24. Retrieve all users sorted alphabetically by name.

```
MariaDB [event_management1]> SELECT * FROM users ORDER BY full_name ASC;
+---------+----------------+------------------+------------+---------+---------------------+
| user_id | full_name      | email            | phone      | city    | created_at          |
+---------+----------------+------------------+------------+---------+---------------------+
|       6 | Aditya Singh   | aditya@gmail.com | 9991110006 | Noida   | 2026-03-26 18:20:38 |
|       2 | Aman Gupta     | aman@gmail.com   | 9991110002 | Delhi   | 2026-03-26 18:20:38 |
|       1 | Kushagra Goyal | kush@gmail.com   | 9991110001 | Noida   | 2026-03-26 18:20:38 |
|       7 | Neha Arora     | neha@gmail.com   | 9991110007 | Delhi   | 2026-03-26 18:20:38 |
|       3 | Priya Sharma   | priya@gmail.com  | 9991110003 | Delhi   | 2026-03-26 18:20:38 |
|      10 | Rahul Mehta    | rahul@gmail.com  | 9991110010 | Delhi   | 2026-03-26 18:20:38 |
|       4 | Rohit Verma    | rohit@gmail.com  | 9991110004 | Gurgaon | 2026-03-26 18:20:38 |
|       9 | Simran Kaur    | simran@gmail.com | 9991110009 | Punjab  | 2026-03-26 18:20:38 |
|       5 | Sneha Kapoor   | sneha@gmail.com  | 9991110005 | Delhi   | 2026-03-26 18:20:38 |
|       8 | Vikas Jain     | vikas@gmail.com  | 9991110008 | Delhi   | 2026-03-26 18:20:38 |
+---------+----------------+------------------+------------+---------+---------------------+
10 rows in set (0.001 sec)
```

Q25. Retrieve the first 5 events from the events table.

```
MariaDB [event_management1]> SELECT * FROM events LIMIT 5;
+----------+----------------+-------------+--------------+----------+------------+------------+----------+-------------+-------------+--------------+----------+
| event_id | event_name     | category_id | organizer_id | venue_id | event_date | start_time | end_time | description | ticket_price | max_capacity | status   |
+----------+----------------+-------------+--------------+----------+------------+------------+----------+-------------+-------------+--------------+----------+
|        1 | Rock Concert   |           1 |            1 |        1 | 2026-04-01 | 18:00:00   | 22:00:00 | NULL        |      999.00 |          500 | Upcoming |
|        2 | Hackathon      |           2 |            2 |        2 | 2026-04-05 | 09:00:00   | 18:00:00 | NULL        |        0.00 |          300 | Upcoming |
|        3 | Football Match |           3 |            3 |        3 | 2026-04-10 | 16:00:00   | 19:00:00 | NULL        |      500.00 |         1000 | Upcoming |
|        4 | Workshop AI    |           4 |            4 |        4 | 2026-04-12 | 10:00:00   | 14:00:00 | NULL        |      200.00 |          400 | Upcoming |
|        5 | Art Expo       |           5 |            5 |        5 | 2026-04-15 | 11:00:00   | 17:00:00 | NULL        |      150.00 |          200 | Upcoming |
+----------+----------------+-------------+--------------+----------+------------+------------+----------+-------------+-------------+--------------+----------+
5 rows in set (0.001 sec)
```

Q26. Retrieve top 3 highest payments.

```
MariaDB [event_management1]> SELECT * FROM payments ORDER BY amount DESC LIMIT 3;
+------------+--------+---------+---------------------+--------------+----------------+---------+
| payment_id | reg_id | amount  | payment_date        | payment_mode | transaction_id | status  |
+------------+--------+---------+---------------------+--------------+----------------+---------+
|          1 |      1 | 1998.00 | 2026-03-26 18:21:09 | UPI          | TXN001         | Success |
|          9 |      9 | 1600.00 | 2026-03-26 18:21:09 | UPI          | TXN009         | Success |
|          3 |      3 | 1500.00 | 2026-03-26 18:21:09 | Card         | TXN003         | Failed  |
+------------+--------+---------+---------------------+--------------+----------------+---------+
3 rows in set (0.001 sec)
```

Q27. Retrieve the event(s) with the highest ticket price.

```
MariaDB [event_management1]> SELECT * FROM events
    -> WHERE ticket_price = (SELECT MAX(ticket_price) FROM events);
+----------+-------------+-------------+--------------+----------+------------+------------+----------+-------------+-------------+--------------+----------+
| event_id | event_name  | category_id | organizer_id | venue_id | event_date | start_time | end_time | description | ticket_price | max_capacity | status   |
+----------+-------------+-------------+--------------+----------+------------+------------+----------+-------------+-------------+--------------+----------+
|        1 | Rock Concert |           1 |            1 |        1 | 2026-04-01 | 18:00:00   | 22:00:00 | NULL        |      999.00 |          500 | Upcoming |
+----------+-------------+-------------+--------------+----------+------------+------------+----------+-------------+-------------+--------------+----------+
1 row in set (0.002 sec)
```

Q28. Retrieve users who have registered for at least one event.

```
MariaDB [event_management1]> SELECT * FROM users
    -> WHERE user_id IN (SELECT user_id FROM registrations);
+---------+----------------+-------------------+-------------+----------+---------------------+
| user_id | full_name      | email             | phone       | city     | created_at          |
+---------+----------------+-------------------+-------------+----------+---------------------+
|       1 | Kushagra Goyal | kush@gmail.com    | 9991110001 | Noida    | 2026-03-26 18:20:38 |
|       2 | Aman Gupta     | aman@gmail.com    | 9991110002 | Delhi    | 2026-03-26 18:20:38 |
|       3 | Priya Sharma   | priya@gmail.com   | 9991110003 | Delhi    | 2026-03-26 18:20:38 |
|       4 | Rohit Verma    | rohit@gmail.com   | 9991110004 | Gurgaon  | 2026-03-26 18:20:38 |
|       5 | Sneha Kapoor   | sneha@gmail.com   | 9991110005 | Delhi    | 2026-03-26 18:20:38 |
|       6 | Aditya Singh   | aditya@gmail.com  | 9991110006 | Noida    | 2026-03-26 18:20:38 |
|       7 | Neha Arora     | neha@gmail.com    | 9991110007 | Delhi    | 2026-03-26 18:20:38 |
|       8 | Vikas Jain     | vikas@gmail.com   | 9991110008 | Delhi    | 2026-03-26 18:20:38 |
|       9 | Simran Kaur    | simran@gmail.com  | 9991110009 | Punjab   | 2026-03-26 18:20:38 |
|      10 | Rahul Mehta    | rahul@gmail.com   | 9991110010 | Delhi    | 2026-03-26 18:20:38 |
+---------+----------------+-------------------+-------------+----------+---------------------+
10 rows in set (0.013 sec)
```

Q29. Retrieve events where more than 2 tickets have been booked.

```
MariaDB [event_management1]> SELECT * FROM events
    -> WHERE event_id IN (SELECT event_id FROM registrations WHERE ticket_count > 2);
+----------+----------------+-------------+--------------+----------+------------+------------+----------+-------------+-------------+--------------+----------+
| event_id | event_name     | category_id | organizer_id | venue_id | event_date | start_time | end_time | description | ticket_price | max_capacity | status   |
+----------+----------------+-------------+--------------+----------+------------+------------+----------+-------------+-------------+--------------+----------+
|        3 | Football Match |           3 |            3 |        3 | 2026-04-10 | 16:00:00   | 19:00:00 | NULL        |      500.00 |         1000 | Upcoming |
|       10 | Food Festival  |          10 |           10 |       10 | 2026-04-28 | 12:00:00   | 22:00:00 | NULL        |      100.00 |          600 | Upcoming |
+----------+----------------+-------------+--------------+----------+------------+------------+----------+-------------+-------------+--------------+----------+
2 rows in set (0.001 sec)
```

Q30. Retrieve payments greater than the average payment amount.

```
MariaDB [event_management1]> SELECT * FROM payments
    -> WHERE amount > (SELECT AVG(amount) FROM payments);
+------------+--------+---------+---------------------+--------------+----------------+---------+
| payment_id | reg_id | amount  | payment_date        | payment_mode | transaction_id | status  |
+------------+--------+---------+---------------------+--------------+----------------+---------+
|          1 |      1 | 1998.00 | 2026-03-26 18:21:09 | UPI          | TXN001         | Success |
|          3 |      3 | 1500.00 | 2026-03-26 18:21:09 | Card         | TXN003         | Failed  |
|          9 |      9 | 1600.00 | 2026-03-26 18:21:09 | UPI          | TXN009         | Success |
+------------+--------+---------+---------------------+--------------+----------------+---------+
3 rows in set (0.002 sec)
```

Q31. Display each event along with the total number of registrations.

```
MariaDB [event_management1]> SELECT e.event_name, COUNT(r.reg_id) AS total_registrations
    -> FROM events e
    -> LEFT JOIN registrations r ON e.event_id = r.event_id
    -> GROUP BY e.event_name;
+-------------------+---------------------+
| event_name        | total_registrations |
+-------------------+---------------------+
| Art Expo          |                   1 |
| Fashion Show      |                   1 |
| Food Festival     |                   1 |
| Football Match    |                   1 |
| Gaming Tournament |                   1 |
| Hackathon         |                   1 |
| Rock Concert      |                   1 |
| Startup Meet      |                   1 |
| Workshop AI       |                   1 |
| Yoga Camp         |                   1 |
+-------------------+---------------------+
10 rows in set (0.002 sec)
```

Q32. Display each user along with the total amount they have spent.

```
MariaDB [event_management1]> SELECT u.full_name, SUM(p.amount) AS total_spent
    -> FROM users u
    -> JOIN registrations r ON u.user_id = r.user_id
    -> JOIN payments p ON r.reg_id = p.reg_id
    -> GROUP BY u.full_name;
+----------------+-------------+
| full_name      | total_spent |
+----------------+-------------+
| Aditya Singh   |      300.00 |
| Aman Gupta     |        0.00 |
| Kushagra Goyal |     1998.00 |
| Neha Arora     |      200.00 |
| Priya Sharma   |     1500.00 |
| Rahul Mehta    |      400.00 |
| Rohit Verma    |      200.00 |
| Simran Kaur    |     1600.00 |
| Sneha Kapoor   |      300.00 |
| Vikas Jain     |      250.00 |
+----------------+-------------+
10 rows in set (0.002 sec)
```

Q33. Retrieve events that have no registrations.

```
MariaDB [event_management1]> SELECT e.event_name
    -> FROM events e
    -> WHERE NOT EXISTS (
    ->     SELECT * FROM registrations r WHERE r.event_id = e.event_id
    -> );
Empty set (0.001 sec)
```

Q34. Display all unique cities where venues are located.

```
MariaDB [event_management1]> SELECT DISTINCT city FROM venues;
+---------+
| city    |
+---------+
| Delhi   |
| Noida   |
| Gurgaon |
+---------+
3 rows in set (0.001 sec)
```

Q35. Retrieve events happening between 1st April 2026 and 20th April 2026.

```
MariaDB [event_management1]> SELECT * FROM events
    -> WHERE event_date BETWEEN '2026-04-01' AND '2026-04-20';
+----------+----------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
| event_id | event_name     | category_id | organizer_id | venue_id | event_date | start_time | end_time | description | ticket_price | max_capacity | status   |
+----------+----------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
|        1 | Rock Concert   |           1 |            1 |        1 | 2026-04-01 | 18:00:00   | 22:00:00 | NULL        |       999.00 |          500 | Upcoming |
|        2 | Hackathon      |           2 |            2 |        2 | 2026-04-05 | 09:00:00   | 18:00:00 | NULL        |         0.00 |          300 | Upcoming |
|        3 | Football Match |           3 |            3 |        3 | 2026-04-10 | 16:00:00   | 19:00:00 | NULL        |       500.00 |         1000 | Upcoming |
|        4 | Workshop AI    |           4 |            4 |        4 | 2026-04-12 | 10:00:00   | 14:00:00 | NULL        |       200.00 |          400 | Upcoming |
|        5 | Art Expo       |           5 |            5 |        5 | 2026-04-15 | 11:00:00   | 17:00:00 | NULL        |       150.00 |          200 | Upcoming |
|        6 | Startup Meet   |           6 |            6 |        6 | 2026-04-18 | 10:00:00   | 15:00:00 | NULL        |       300.00 |          350 | Upcoming |
|        7 | Yoga Camp      |           7 |            7 |        7 | 2026-04-20 | 06:00:00   | 09:00:00 | NULL        |       100.00 |          250 | Upcoming |
+----------+----------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
7 rows in set (0.001 sec)
```

Q36. Use a LEFT JOIN to list all venues and the events hosted at each.

```
MariaDB [event_management1]> SELECT e.event_name, v.city
    -> FROM events e
    -> JOIN venues v ON e.venue_id = v.venue_id
    -> WHERE v.city = 'Delhi';
+--------------------+-------+
| event_name         | city  |
+--------------------+-------+
| Rock Concert       | Delhi |
| Football Match     | Delhi |
| Workshop AI        | Delhi |
| Art Expo           | Delhi |
| Yoga Camp          | Delhi |
| Gaming Tournament  | Delhi |
| Fashion Show       | Delhi |
| Food Festival      | Delhi |
+--------------------+-------+
8 rows in set (0.001 sec)
```

Q37. Find the second most recent payment transaction.

```
MariaDB [event_management1]> SELECT * FROM payments WHERE status = 'Failed';
+------------+--------+---------+---------------------+--------------+----------------+--------+
| payment_id | reg_id | amount  | payment_date        | payment_mode | transaction_id | status |
+------------+--------+---------+---------------------+--------------+----------------+--------+
|          3 |      3 | 1500.00 | 2026-03-26 18:21:09 | Card         | TXN003         | Failed |
|          8 |      8 |  250.00 | 2026-03-26 18:21:09 | Card         | TXN008         | Failed |
+------------+--------+---------+---------------------+--------------+----------------+--------+
2 rows in set (0.001 sec)
```

Q38. Retrieve users who have not registered for any event.

```
MariaDB [event_management1]> SELECT u.full_name
    -> FROM users u
    -> WHERE u.user_id NOT IN (SELECT user_id FROM registrations);
Empty set (0.002 sec)
```

Q39. Display events where total tickets booked are greater than 2.

```
MariaDB [event_management1]> SELECT e.event_name, SUM(r.ticket_count) AS total_tickets
    -> FROM events e
    -> JOIN registrations r ON e.event_id = r.event_id
    -> GROUP BY e.event_name
    -> HAVING total_tickets > 2;
+----------------+---------------+
| event_name     | total_tickets |
+----------------+---------------+
| Food Festival  |             4 |
| Football Match |             3 |
+----------------+---------------+
2 rows in set (0.001 sec)
```

Q40. Retrieve all upcoming events.

```
MariaDB [event_management1]> SELECT * FROM events WHERE status = 'Upcoming';
+----------+------------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
| event_id | event_name       | category_id | organizer_id | venue_id | event_date | start_time | end_time | description | ticket_price | max_capacity | status   |
+----------+------------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
|        1 | Rock Concert     |           1 |            1 |        1 | 2026-04-01 | 18:00:00   | 22:00:00 | NULL        |       999.00 |          500 | Upcoming |
|        2 | Hackathon        |           2 |            2 |        2 | 2026-04-05 | 09:00:00   | 18:00:00 | NULL        |         0.00 |          300 | Upcoming |
|        3 | Football Match   |           3 |            3 |        3 | 2026-04-10 | 16:00:00   | 19:00:00 | NULL        |       500.00 |         1000 | Upcoming |
|        4 | Workshop AI      |           4 |            4 |        4 | 2026-04-12 | 10:00:00   | 14:00:00 | NULL        |       200.00 |          400 | Upcoming |
|        5 | Art Expo         |           5 |            5 |        5 | 2026-04-15 | 11:00:00   | 17:00:00 | NULL        |       150.00 |          200 | Upcoming |
|        6 | Startup Meet     |           6 |            6 |        6 | 2026-04-18 | 10:00:00   | 15:00:00 | NULL        |       300.00 |          350 | Upcoming |
|        7 | Yoga Camp        |           7 |            7 |        7 | 2026-04-20 | 06:00:00   | 09:00:00 | NULL        |       100.00 |          250 | Upcoming |
|        8 | Gaming Tournament |          8 |            8 |        8 | 2026-04-22 | 12:00:00   | 20:00:00 | NULL        |       250.00 |          150 | Upcoming |
|        9 | Fashion Show     |           9 |            9 |        9 | 2026-04-25 | 17:00:00   | 21:00:00 | NULL        |       800.00 |          300 | Upcoming |
|       10 | Food Festival    |          10 |           10 |       10 | 2026-04-28 | 12:00:00   | 22:00:00 | NULL        |       100.00 |          600 | Upcoming |
+----------+------------------+-------------+--------------+----------+------------+------------+----------+-------------+--------------+--------------+----------+
10 rows in set (0.001 sec)
```

Database Normalization

Database	Tables	Normal Forms Covered
event_management1	categories, events, organizers, payments, registrations, users, venues	1NF → 2NF → 3NF → BCNF

1. Introduction to Normalization

Database normalization organizes a relational database to reduce data redundancy and improve data integrity. It eliminates Insertion Anomalies, Update Anomalies, and Deletion Anomalies through a series of stages called Normal Forms (NF).

Normal Form	Requirement
1NF	Each column holds atomic (indivisible) values; every row is unique; no repeating groups.
2NF	1NF + no partial dependency — every non-key attribute depends on the whole primary key.
3NF	2NF + no transitive dependency — non-key attributes depend only on the primary key.
BCNF	Stricter 3NF — every determinant must be a candidate key (superkey).

2. Original Schema

Table	Primary Key	Columns
categories	category_id	category_name, description
events	event_id	event_name, category_id (FK), organizer_id (FK), venue_id (FK), event_date, start_time, end_time, description, ticket_price, max_capacity, status
organizers	organizer_id	org_name, contact_person, email, phone, website
payments	payment_id	reg_id (FK), amount, payment_date, payment_mode, transaction_id, status
registrations	reg_id	user_id (FK), event_id (FK), reg_date, ticket_count, status
users	user_id	full_name, email, phone, city, created_at
venues	venue_id	venue_name, address, city, capacity, contact_phone

3. First Normal Form (1NF)

Rules: (1) All column values are atomic. (2) Each column has a unique name. (3) Each row is uniquely identifiable by a primary key. (4) No repeating groups of columns.

Table	Atomic Values?	Unique Column Names?	PK Exists?	No Repeating Groups?	Status
categories	Yes	Yes	Yes (category_id)	Yes	✔ Satisfies 1NF
events	Yes	Yes	Yes (event_id)	Yes	✔ Satisfies 1NF
organizers	Yes	Yes	Yes (organizer_id)	Yes	✔ Satisfies 1NF
payments	Yes	Yes	Yes (payment_id)	Yes	✔ Satisfies 1NF

Table	Atomic Values?	Unique Column Names?	PK Exists?	No Repeating Groups?	Status
registrations	Yes	Yes	Yes (reg_id)	Yes	✔ Satisfies 1NF
users	Yes	Yes	Yes (user_id)	Yes	✔ Satisfies 1NF
venues	Yes	Yes	Yes (venue_id)	Yes	✔ Satisfies 1NF

1NF Conclusion: All seven tables satisfy First Normal Form. Every attribute stores a single atomic value, column names are unique within each table, and primary keys ensure row uniqueness.

4. Second Normal Form (2NF)

Rule: Table must be in 1NF AND every non-key attribute must be fully functionally dependent on the entire primary key. Partial dependencies can only occur with composite primary keys. A single-column primary key automatically eliminates the possibility of partial dependencies.

Table	Primary Key	Key Type	Partial Dependency Possible?	Status
categories	category_id	Single	No	✔ Satisfies 2NF
events	event_id	Single	No	✔ Satisfies 2NF
organizers	organizer_id	Single	No	✔ Satisfies 2NF
payments	payment_id	Single	No	✔ Satisfies 2NF
registrations	reg_id	Single	No	✔ Satisfies 2NF
users	user_id	Single	No	✔ Satisfies 2NF
venues	venue_id	Single	No	✔ Satisfies 2NF

2NF Conclusion: All seven tables use single-column surrogate primary keys. Since no table has a composite primary key, partial dependencies cannot exist. All tables satisfy 2NF.

5. Third Normal Form (3NF)

Rule: Table must be in 2NF AND no non-key attribute should transitively depend on another non-key attribute. For every FD $X \rightarrow Y$, X must be a superkey or Y must be part of a candidate key.

Table	Non-Key Attributes	Transitive Dependency?	Status
categories	category_name, description	None — both depend directly on category_id	✔ Satisfies 3NF
events	event_name, event_date, start_time, end_time, ticket_price, max_capacity, status	None — all attributes are properties of the event itself; FK columns (category_id, organizer_id, venue_id) are references, not derived attributes	✔ Satisfies 3NF
organizers	org_name, contact_person, email, phone, website	None — all depend directly on organizer_id	✔ Satisfies 3NF

Table	Non-Key Attributes	Transitive Dependency?	Status
payments	reg_id, amount, payment_date, payment_mode, transaction_id, status	amount could equal ticket_count × ticket_price (from registrations × events). If amount is always derived, it is a transitive dependency. Treating amount as an authoritative stored value (to allow discounts/fees) resolves this.	■ Borderline — amount must be authoritative, not derived
registrations	user_id, event_id, reg_date, ticket_count, status	None — user_id and event_id are FK references; reg_date, ticket_count, status are direct properties of the registration	✔ Satisfies 3NF
users	full_name, email, phone, city, created_at	None — city is a direct user attribute; no state/country derived from city	✔ Satisfies 3NF
venues	venue_name, address, city, capacity, contact_phone	None — all depend directly on venue_id	✔ Satisfies 3NF

3NF Conclusion: Six out of seven tables fully satisfy 3NF. The payments table has a borderline case: if amount is always equal to ticket_count × ticket_price, it constitutes a transitive dependency. This is resolved by treating amount as an authoritative stored value that can accommodate discounts and fees.

6. Boyce–Codd Normal Form (BCNF)

Rule: For every non-trivial functional dependency X → Y, X must be a superkey. BCNF is stricter than 3NF and handles anomalies involving overlapping candidate keys.

Table	Primary Key	Other Candidate Keys	All Determinants Superkeys?	Status
categories	category_id	None identified	Yes	✔ BCNF
events	event_id	None identified	Yes	✔ BCNF
organizers	organizer_id	email (all values distinct)	Yes — email is a superkey	✔ BCNF
payments	payment_id	transaction_id (all values distinct)	Yes — transaction_id is a superkey	✔ BCNF
registrations	reg_id	None identified	Yes	✔ BCNF
users	user_id	email (all values distinct)	Yes — email is a superkey	✔ BCNF
venues	venue_id	contact_phone (all values distinct)	Yes — contact_phone is a superkey	✔ BCNF

BCNF Conclusion: All tables satisfy BCNF. In tables where additional unique columns exist (email in users/organizers, transaction_id in payments, contact_phone in venues), they act as candidate keys (superkeys), so BCNF is satisfied for all non-trivial functional dependencies.

7. Anomaly Analysis

Anomaly	Description	How the Schema Handles It	Status
Insertion Anomaly	Cannot insert data without another entity's data.	Separate tables allow independent insertion — e.g., a venue can be added without any event.	✔ Resolved

Anomaly	Description	How the Schema Handles It	Status
Update Anomaly	Updating one fact requires changes in multiple rows.	Organizer details live in the organizers table; changing an email requires updating one row only.	✔ Resolved
Deletion Anomaly	Deleting a record unintentionally removes other data.	Deleting a registration does not delete the user or event. FK relationships preserve referential integrity.	✔ Resolved

8. Schema Relationships (Foreign Keys)

Foreign Key	References	Relationship
events.category_id	categories.category_id	Many events → One category
events.organizer_id	organizers.organizer_id	Many events → One organizer
events.venue_id	venues.venue_id	Many events → One venue
registrations.user_id	users.user_id	Many registrations → One user
registrations.event_id	events.event_id	Many registrations → One event
payments.reg_id	registrations.reg_id	One payment → One registration

9. Recommendations

#	Column / Concern	Recommendation
R1	payments.amount	Treat as an authoritative stored value to allow discounts and fees. Add a comment or constraint documenting this decision to prevent future confusion with derived values.
R2	events.max_capacity vs venues.capacity	Add a CHECK constraint or trigger to ensure events.max_capacity ≤ venues.capacity.
R3	Unique constraints	Explicitly declare UNIQUE on: users.email, organizers.email, payments.transaction_id, venues.contact_phone to enforce the candidate keys identified in the BCNF analysis.
R4	events.description	Column appears NULL in sample data. Add a NOT NULL constraint or a default placeholder value.
R5	FK index coverage	Add indexes on events.category_id, events.organizer_id, events.venue_id, registrations.user_id, registrations.event_id, payments.reg_id to improve JOIN query performance.

10. Normalization Summary

Table	1NF	2NF	3NF	BCNF	Remarks
categories	✔	✔	✔	✔	Fully normalised
events	✔	✔	✔	✔	Fully normalised
organizers	✔	✔	✔	✔	Fully normalised
payments	✔	✔	■	✔	amount must be treated as authoritative, not derived
registrations	✔	✔	✔	✔	Fully normalised
users	✔	✔	✔	✔	Fully normalised

Table	1NF	2NF	3NF	BCNF	Remarks
venues	✔	✔	✔	✔	Fully normalised

Overall Conclusion: The event_management1 database is well-designed and satisfies 1NF, 2NF, 3NF, and BCNF for all practical purposes. The single borderline concern is payments.amount, which may represent a transitive dependency if always derivable from ticket_count × ticket_price. Applying the five recommendations above will make the schema robustly normalised and production-ready.